

Falcon: A Reliable, Low Latency Hardware Transport

Professional Research Breakdown

Venue: ACM SIGCOMM 2025 **DOI:** [10.1145/3718958.3754353](https://doi.org/10.1145/3718958.3754353)

Pages: 248–263 **Published online:** 2025-08-27

Prepared for Ethan by Ditto

Source note: I verified the paper metadata through the SIGCOMM 2025 accepted-papers page, Crossref, DBLP, Unpaywall, and Semantic Scholar. The ACM PDF endpoint is open-access according to Unpaywall, but direct automated retrieval from ACM was blocked by Cloudflare in this environment. This report is therefore grounded in the verified title, author list, venue metadata, abstract, and public bibliographic records; detailed claims below are explicitly marked when they are inferred from the abstract and from standard datacenter transport design principles rather than from full-paper sections.

Bibliographic Snapshot

- **Paper:** *Falcon: A Reliable, Low Latency Hardware Transport*.
- **Authors:** Arjun Singhvi, Nandita Dukkupati, Prashant Chandra, Hassan M. G. Wassel, Naveen Kr. Sharma, Anthony Rebello, Henry Schuh, Praveen Kumar, Behnam Montazeri, Neelesh Bansod, Sarin Thomas, Inho Cho, Hyojeong Lee Seibert, Baijun Wu, Rui Yang, Yuliang Li, Kai Huang, Qianwen Yin, Abhishek Agarwal, Srinivas Vaduvatha, Weihuang Wang, Masoud Moshref, Tao Ji, David Wetherall, Amin Vahdat.
- **Affiliations listed on SIGCOMM page:** primarily Google / Google LLC, with contributors from Meta, Nvidia, and Microsoft.
- **Venue:** Proceedings of the ACM SIGCOMM 2025 Conference.
- **Core abstract claim:** Falcon is a hardware transport for general-purpose Ethernet datacenter environments with losses and without special switch support; it supports multiple upper-layer protocols and heterogeneous application workloads.

Executive Summary

Falcon targets a gap between two worlds. On one side, **RoCE** provides very high performance with low CPU overhead, but it is typically deployed in carefully engineered environments: backend networks, lossless or near-lossless Ethernet, Priority Flow Control, and limited application diversity. On the other side, general-purpose datacenter Ethernet is messier: packet loss exists, workloads are heterogeneous, application protocols vary, and relying on special switch features is operationally expensive.

The paper’s central contribution is a **hardware transport** that attempts to keep the performance profile of RDMA-like transports while making the deployment model closer to ordinary lossy Ethernet. The abstract names four major design ingredients:

1. **Delay-based congestion control with multipath load balancing.**
2. **A layered design exposing a simple request-response transaction interface** so multiple upper-layer protocols can share the transport.
3. **Hardware-based retransmissions and error handling** for scalability and CPU efficiency.
4. **A programmable engine** to retain flexibility despite hardware implementation.

The first hardware implementation reportedly reaches **200 Gbps peak performance** and **120 million operations per second**. Under congestion, operation completion time is reported as up to **8 times lower than CX-7 RoCE**; under lossy conditions, goodput is up to **65% higher**. The paper is therefore positioned not as “yet another software transport”, but as a serious hardware datapath alternative for reliable, low-latency RPC/RDMA-like communication in production Ethernet fabrics.

Problem: Why Existing Hardware Transports Are Not Enough

The RoCE trade-off

RoCE is attractive because it bypasses the host CPU and gives applications low-latency, high-throughput remote memory or messaging semantics. But RoCE’s best-known operational pain point is that it is commonly paired with lossless or carefully controlled Ethernet behavior. In practice, operators often rely on Priority Flow Control, traffic isolation, and careful queue management. This works well for specialized backend deployments, but it is harder to make universal across heterogeneous workloads.

The paper’s motivation can be summarized as:

Can we get hardware-transport efficiency without requiring a special-purpose, lossless network environment?

This question matters because large datacenters increasingly run many traffic classes over shared fabrics: storage, RPCs, AI training/inference communication, distributed databases, telemetry, and background transfers. A transport that only behaves well under special conditions is powerful but operationally narrow.

What changes in general-purpose Ethernet

General-purpose Ethernet introduces several stressors:

- **Loss is real.** Congestion drops, transient microbursts, buffer pressure, link events, and ECMP imbalance can all cause loss.
- **Switch assistance is not guaranteed.** The abstract explicitly says Falcon avoids special switch support, which implies the endpoints must absorb more responsibility.
- **Workloads are heterogeneous.** A single transport may need to support RPC-like messaging, storage-style operations, remote memory access, or other upper-layer protocol semantics.
- **Tail latency matters.** Hardware transport users typically care not just about peak bandwidth, but also operation completion time under congestion and loss.

For Ethernet-scale AI and cloud infrastructure, this is exactly the uncomfortable zone: the network is fast, but not perfectly lossless; the hardware path must be efficient, but rigid protocols age badly; and transport behavior under incast, multipath imbalance, or transient congestion is often the difference between a clean SLO and a production boss fight.

Core Idea and Design Mechanism

Falcon as a hardware transport substrate

The phrase “hardware transport” is important. Falcon is not described as a host-stack software protocol. The goal is to move reliability, congestion response, retransmission, and transaction handling into hardware so that applications obtain low latency and low CPU overhead.

The design appears to expose a **request-response transaction interface**. That is a meaningful abstraction choice. Instead of binding the hardware tightly to one upper-layer protocol, Falcon provides a common transaction substrate that multiple ULPs can map onto. This helps explain the abstract's claim that Falcon supports multiple ULPs and heterogeneous workloads.

Delay-based congestion control

The abstract explicitly identifies delay-based congestion control as a key element. The likely design intent is to react before queues build to loss. Compared with purely loss-based control, delay signals can detect congestion earlier. Compared with pure rate-driven schemes, delay feedback can be useful in shallow-buffer, low-latency datacenter environments where waiting for drops is too slow and damaging.

For a hardware transport, delay-based control is especially appealing because it can be implemented as a fast feedback loop at the endpoint. The challenge is robustness: delay signals are noisy, affected by multipath variation, endpoint scheduling, and queueing outside the network. A good Falcon design therefore likely needs filtering, stable control laws, and coordination with multipath load balancing.

Multipath load balancing

The abstract pairs congestion control with multipath load balancing. This is a big clue about the target environment: Falcon is intended for modern datacenter fabrics where path diversity exists but can be unevenly loaded. Multipath transport can reduce hot-spotting and improve goodput, but it creates its own problems:

- packets may arrive out of order;
- paths may have different delay baselines;
- congestion estimates can become path-specific rather than connection-wide;
- retransmission and completion ordering become more complex in hardware.

A professional reading is that Falcon's real difficulty is not any single mechanism. It is the combination: reliable transaction processing, low latency, loss tolerance, multipath, and hardware offload, all while supporting multiple ULPs.

Hardware retransmissions and error handling

The abstract highlights hardware-based retransmissions and error handling. This is where Falcon likely diverges from many host-centric approaches. Moving retransmission into hardware reduces CPU involvement and can keep recovery fast, which matters for short operations where a single software round trip would dominate latency.

The key systems trade-off is state. Reliable hardware retransmission requires the device to track outstanding operations, sequence or transaction identifiers, timers, buffers, and error states. At scale, that state must fit in limited device resources and must be robust to pathological cases such as receiver stalls, repeated loss, or multipath reordering. This is why the reported **120 Mops/sec** result matters: it suggests the implementation is not just a feature prototype, but a high-rate hardware datapath.

Programmable engine

A fully fixed-function hardware transport risks becoming obsolete as workloads and upper-layer protocols evolve. Falcon addresses this by including a programmable engine. This is important architecturally: the fast path can stay in hardware, while policy or protocol-specific behavior can retain some flexibility.

The design tension is classic SmartNIC/CNIC territory: too much programmability can reduce performance predictability; too little programmability makes the transport hard to adapt. Falcon’s contribution appears to be choosing a boundary where retransmission, transaction processing, and congestion response remain efficient enough for hardware, while still allowing protocol evolution.

How Falcon Compares with RoCE

Dimension	RoCE-style deployment	Falcon’s claimed direction
Network assumption	Often optimized for controlled or loss-less Ethernet; PFC is common in practice.	Designed for general-purpose Ethernet with losses and no special switch support.
Workload scope	Strong for specialized backend/RDMA deployments.	Multiple ULPs and heterogeneous application workloads.
Loss handling	Loss is usually something the environment tries hard to avoid.	Loss is part of the design target; hardware retransmission and error handling are central.
Congestion behavior	Can suffer under congestion or PFC-induced effects depending on configuration.	Delay-based congestion control plus multipath load balancing.
Flexibility	Mature ecosystem, but semantics can be rigid.	Layered transaction interface plus programmable engine.
Reported result	Baseline comparison uses CX-7 RoCE.	Up to 8x lower completion time under congestion; up to 65% higher goodput under loss.

Evaluation Claims and What They Mean

The abstract reports four headline numbers:

- **200 Gbps peak performance.**
- **120 Mops/sec.**
- **Up to 8x lower operation completion time than CX-7 RoCE under network congestion.**
- **Up to 65% higher goodput under lossy conditions.**

Interpreting the 200 Gbps result

A 200 Gbps hardware implementation indicates that Falcon is operating at serious line-rate scale rather than being a control-plane prototype. For a transport paper, this matters because mechanisms that look good in simulation can fail when implemented in hardware due to state pressure, SRAM limits, timer granularity, PCIe interactions, packet parsing constraints, or scheduler design.

Interpreting 120 Mops/sec

Operations per second is especially relevant if Falcon exposes a request-response transaction interface. High Mops/sec suggests the hardware can handle small operations at scale, not merely bulk throughput. That is important for RPC-like workloads, metadata operations, key-value accesses, storage operations, and AI control-plane or serving systems where many transactions are small and latency-sensitive.

Interpreting the congestion result

Up to 8x lower operation completion time under congestion is the most important latency claim. The natural explanation is that delay-based control and multipath load balancing prevent or escape queue buildup and hot-spotting more effectively than the RoCE baseline under the tested conditions. The exact generality depends on workload mix, topology, offered load, path entropy, and baseline RoCE configuration.

Interpreting the loss result

Up to 65% higher goodput under loss suggests Falcon's hardware retransmission/error handling has a meaningful advantage when drops occur. This is a direct challenge to the idea that high-performance hardware transports must rely on lossless behavior. For operators, this is operationally valuable: if the transport tolerates loss gracefully, the network can be simpler and less fragile.

Why This Matters for AI Datacenter Networks

Falcon is not branded as an AI networking paper, but it is highly relevant to AI infrastructure.

AI workloads increasingly need general-purpose reliable low latency

Training and inference systems use multiple communication patterns: collectives, point-to-point tensor transfers, storage reads, KV-cache movement, RPC control traffic, parameter/expert routing, and service orchestration. Not all of these fit one clean RDMA abstraction. A multi-ULP hardware transport is attractive because it could unify several communication substrates while preserving low CPU overhead.

Lossless Ethernet is operationally expensive

Large AI clusters often push operators toward highly engineered fabrics. Lossless behavior can help performance but introduces risks: head-of-line blocking, pause storms, complicated QoS interactions, and fragile tuning. A reliable hardware transport that works over lossy Ethernet could reduce dependence on network-wide losslessness. That is a very meaningful direction for large superpods where topology and workload diversity make perfect isolation difficult.

Multipath matters more as clusters scale

As AI clusters scale, path diversity and load imbalance become central. Static hashing can create hot spots, and large flows can dominate paths. Falcon's coupling of multipath load balancing with congestion control is relevant because AI systems increasingly need transport mechanisms that are topology-aware, congestion-sensitive, and fast enough for short operations.

Programmability is a hedge against workload drift

AI infrastructure changes quickly. A fixed protocol designed around one workload may age poorly. Falcon's programmable engine suggests a design philosophy similar to modern SmartNIC/CNIC systems: keep the common fast path in hardware, but retain enough programmability to adapt policy and protocol behavior.

Strengths

- **Clear operational target:** Falcon addresses the deployment pain of RoCE-like transports in general-purpose Ethernet networks.
- **Hardware realism:** The reported implementation metrics indicate a real hardware datapath, not just simulation.
- **Multi-ULP abstraction:** The request-response transaction interface is a clean way to avoid designing one transport per application protocol.
- **Loss tolerance:** Designing for lossy Ethernet is strategically important for large cloud fabrics.
- **Congestion and load-balancing integration:** Delay-based control plus multipath is the right conceptual pairing for low-latency datacenter environments.

Limitations and Questions to Ask When Reading the Full Paper

Because the full PDF was not directly retrievable in this environment, the following are framed as reading questions rather than asserted weaknesses.

1. **What is the exact congestion-control law?** Delay-based schemes live or die by stability, filtering, fairness, and response to noisy delay samples.
2. **How does Falcon isolate traffic classes?** If multiple ULPs share the transport, the scheduler and QoS model become critical.
3. **How much state is required per operation or connection?** Hardware retransmission at 120 Mops/sec implies serious state-management design.
4. **What are the failure modes under severe packet reordering?** Multipath can improve load balancing but complicates ordering and recovery.
5. **How fair is Falcon against TCP, RoCE, or other production traffic?** General-purpose Ethernet means coexistence matters.
6. **How does it behave under incast?** Reliable low-latency transports often struggle when many senders target one receiver.
7. **What switch features are truly unnecessary?** “No special switch support” is strong; it would be useful to know whether ordinary ECMP, ECN, or queue telemetry is assumed.
8. **How programmable is the programmable engine?** The boundary between fixed function and programmable logic determines long-term adaptability.

Practical Takeaways

- **Endpoint intelligence can substitute for network special-casing.** Falcon’s design suggests that smarter hardware endpoints can reduce dependence on PFC and special switch behavior.
- **Loss tolerance is not optional at cloud scale.** Treating loss as a first-class condition rather than an exceptional failure mode makes the transport more deployable.
- **Transaction abstractions are powerful.** A request-response substrate may be easier to share across storage, RPC, and AI infrastructure traffic than a single narrow RDMA semantic.

- **Transport design is becoming CNIC design.** Reliability, congestion control, multipath, retransmission, QoS, and programmability increasingly belong in the NIC or near-NIC datapath.
- **For AI superpods, the interesting question is coexistence.** Falcon-like transports could be valuable for non-collective traffic, KV movement, metadata, and RPCs, but must coexist with collective communication and latency-critical model traffic.

Bottom Line

Falcon is important because it attacks a practical deployment barrier: high-performance hardware transports are compelling, but their operational assumptions are often too narrow. By targeting lossy general-purpose Ethernet, multiple upper-layer protocols, hardware retransmission, delay-based congestion control, multipath load balancing, and programmable hardware flexibility, Falcon points toward a more deployable transport substrate for cloud-scale datacenters.

For AI infrastructure, the paper's broader implication is that future superpod networks may need transports that are not merely fast in ideal conditions, but **reliable, low-latency, loss-tolerant, multipath-aware, and programmable**. That is the direction where Falcon is most interesting.

Verified Source Links

- SIGCOMM 2025 accepted papers: <https://conferences.sigcomm.org/sigcomm/2025/accepted-papers/>
- DOI landing page: <https://doi.org/10.1145/3718958.3754353>
- DBLP record: <https://dblp.org/rec/conf/sigcomm/SinghviDCWSRS0M25.html>
- Semantic Scholar record: <https://www.semanticscholar.org/paper/947b2ae2d65ef98fe31146ae1ec6f617af38ebfd>